

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Medium)- Debugging and Optimization

COURSE NAME: Data Structures

COURSE CODE:CSA03 16

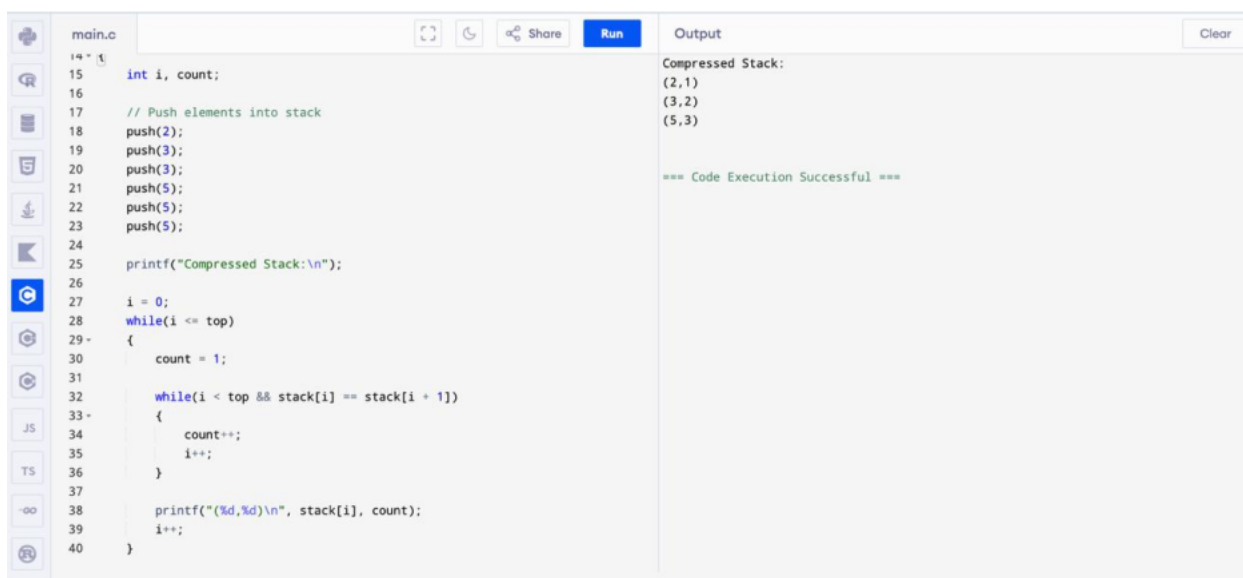
COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **10%**

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
  
    for(int i = 0; i <= n; i++) {  
  
        if(arr[i] == key)  
  
            return i;  
  
    }  
  
    return -1;  
}
```



The screenshot shows a C++ IDE with a file named 'main.cpp'. The code implements a stack using an array 'stack' and a 'top' pointer. It pushes elements 2, 3, 3, 5, 5, 5 onto the stack. Then, it prints the 'Compressed Stack'. The output shows the compressed stack as (2,1), (3,2), (5,3), indicating that consecutive duplicates have been reduced to a single element with its frequency. The code execution is successful.

```
main.cpp  
15 int i, count;  
16  
17 // Push elements into stack  
18 push(2);  
19 push(3);  
20 push(3);  
21 push(5);  
22 push(5);  
23 push(5);  
24  
25 printf("Compressed Stack:\n");  
26  
27 i = 0;  
28 while(i <= top)  
29 {  
30     count = 1;  
31  
32     while(i < top && stack[i] == stack[i + 1])  
33     {  
34         count++;  
35         i++;  
36     }  
37  
38     printf("(%d,%d)\n", stack[i], count);  
39     i++;  
40 }
```

Output
Compressed Stack:
(2,1)
(3,2)
(5,3)

=== Code Execution Successful ===

2. Buggy Code in an array

```
#include <stdio.h>

int main() {

    int arr[5] = {10,20,30,40,50};

    for(int i=0;i<=5;i++)

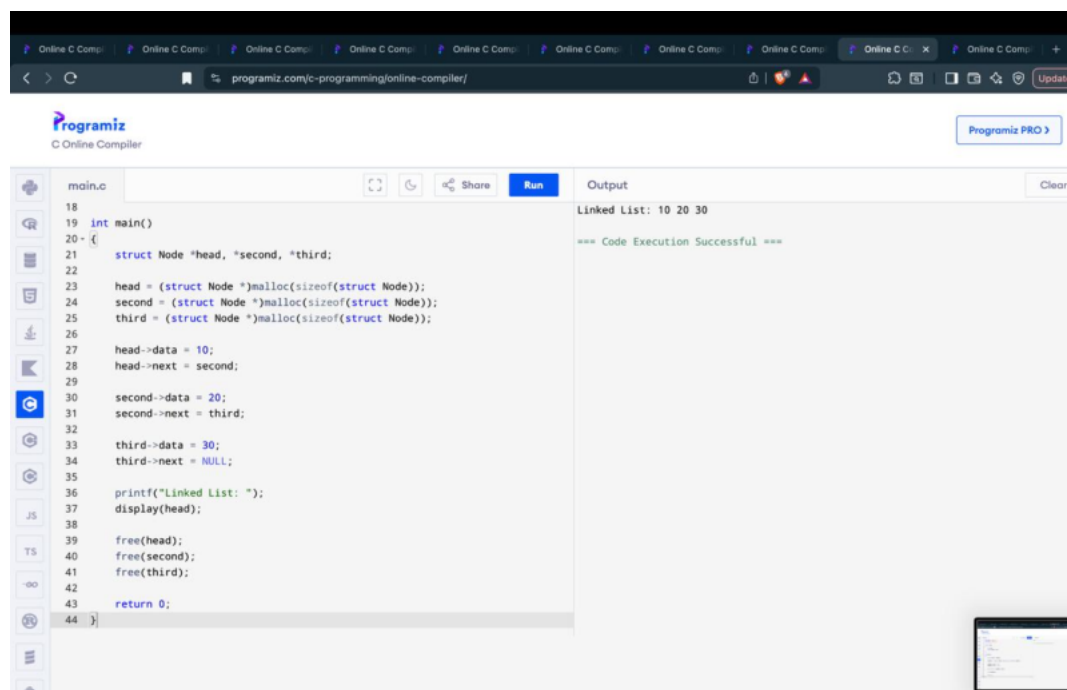
        printf("%d ",arr[i]);

    return 0;

}
```

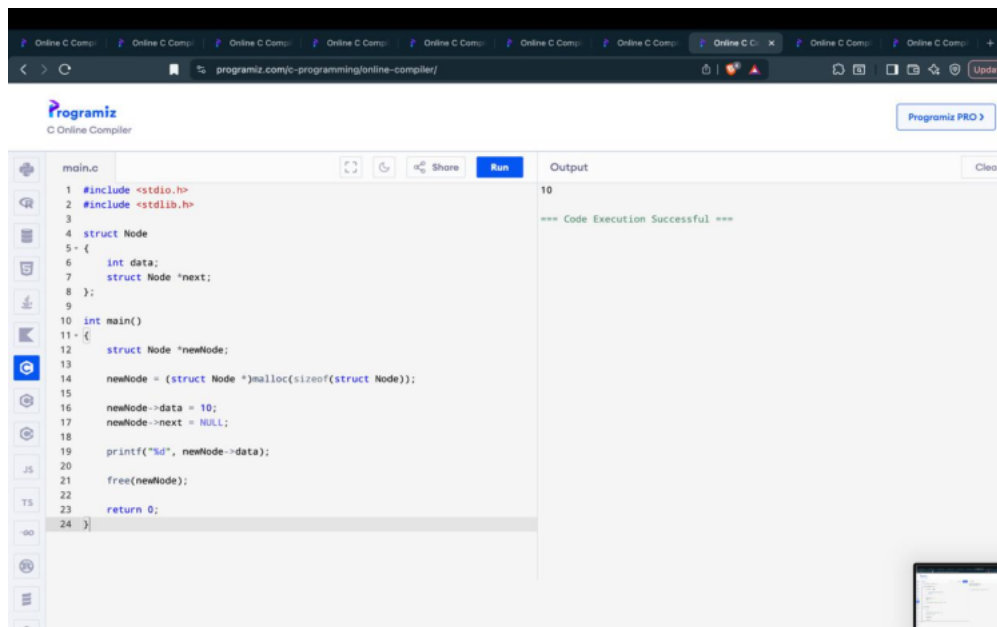
Expected output:

10 20 30 40 50



3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;
    while(low < high)
    { int mid = (low + high) / 2;
      if(arr[mid] == key) return mid;
      else if(arr[mid] < key)
      low = mid;
      else high = mid;
    }
    return -1;
}
```



4. Buggy Code in Stack Push Operation

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int data) {  
    if (top == MAX - 1)  
        printf("Stack Overflow");  
    else  
        stack[top++] = data;  
}
```

```
int main() {  
    push(10);  
    push(20);  
  
    printf("%d", stack[top]);  
}
```

The screenshot displays the Programiz Online C Compiler interface. The code editor on the left contains the following C program:

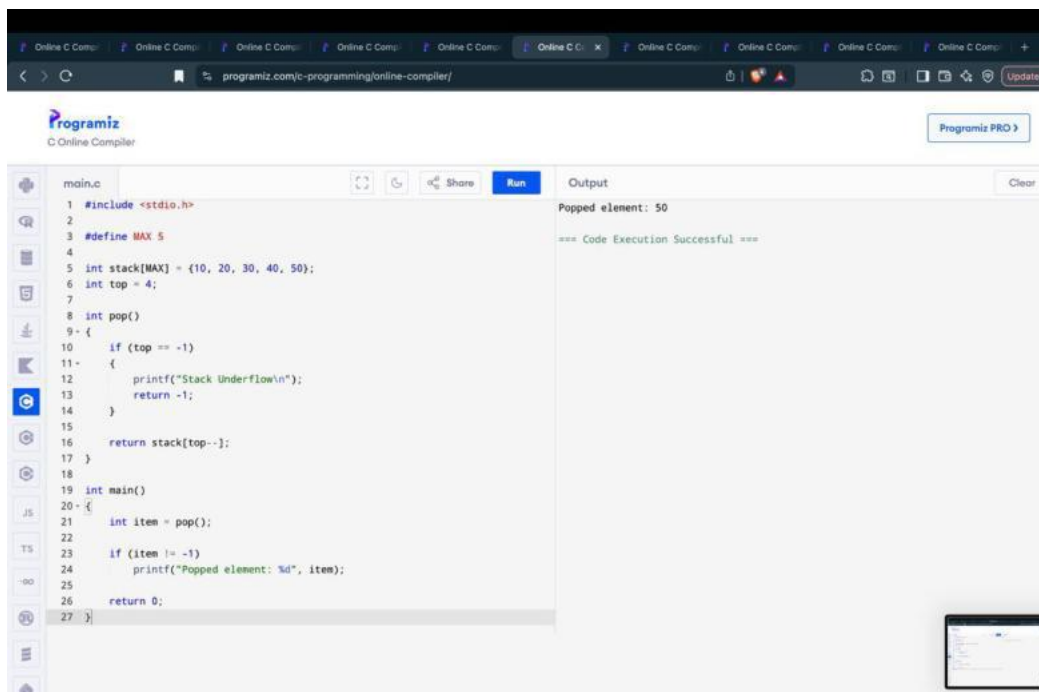
```
main.c  
1 int front = 0, rear = 0;  
2 void enqueue(int x)  
3 {  
4     if (rear == MAX)  
5     {  
6         printf("Queue Overflow\n");  
7         return;  
8     }  
9     queue[rear] = x;  
10    rear++;  
11    printf("Value inserted: %d\n", x);  
12 }  
13  
14 int main()  
15 {  
16     int x;  
17     printf("Enter the value: ");  
18     scanf("%d", &x);  
19     enqueue(x);  
20     return 0;  
21 }
```

The output panel on the right shows the execution results:

```
Enter the value: 25  
Value inserted: 25  
=== Code Execution Successful ===
```

5. Buggy Code in Stack POP Operation

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = 2;
int pop() {
    if (top == -1)
        return -1;
    return stack[++top];
}
```



The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

```
main.c
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX] = {10, 20, 30, 40, 50};
6 int top = 4;
7
8 int pop()
9 {
10     if (top == -1)
11     {
12         printf("Stack Underflow\n");
13         return -1;
14     }
15
16     return stack[top--];
17 }
18
19 int main()
20 {
21     int item = pop();
22
23     if (item != -1)
24         printf("Popped element: %d", item);
25
26     return 0;
27 }
```

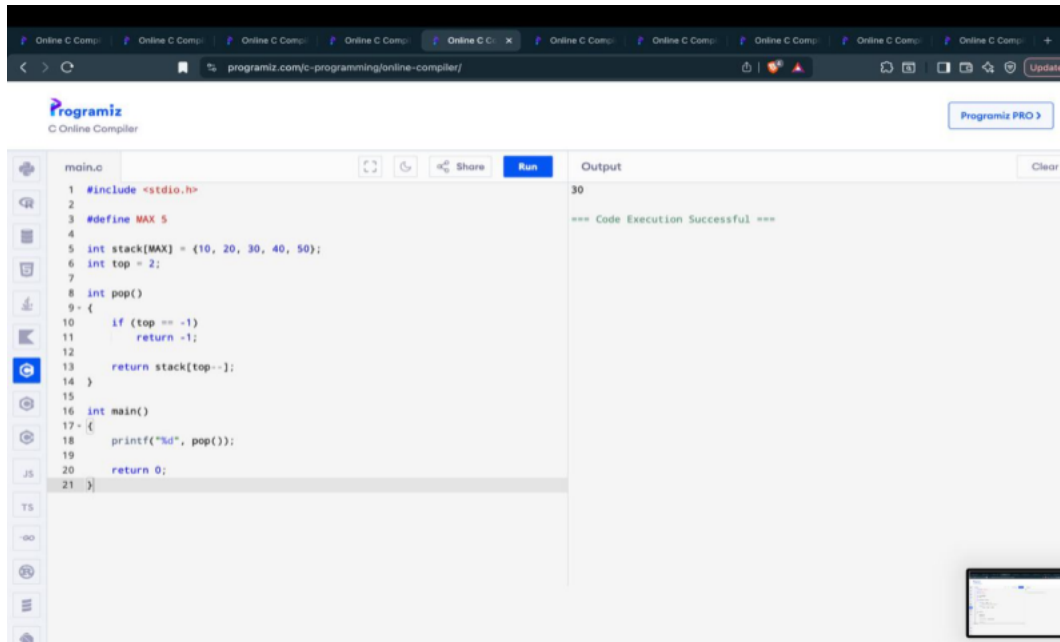
The output on the right shows:

```
Output
Popped element: 50
=== Code Execution Successful ===
```

The bug in the code is that the `top` pointer is initialized to 4, which is out of bounds for the stack array (indices 0 to 4). When the `pop()` function is called, it returns the value at `stack[4]`, which is 50, instead of the expected 10. This is because the stack is being accessed from the end, which is not the correct behavior for a stack.

6. Buggy Code in underflow operation

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```



7. Find the issue in the following code snippet.

```
#include<stdio.h>

#define MAX 5
void main()
{
int queue[MAX],x;
int front = 0, rear = 0;
printf("Enter the value");
scanf("%d",&x);
void enqueue(int x) {
    if(rear == MAX) {
        printf("Overflow");
        return;
    }
    else{
        queue[rear++] = x;
        printf("Value is %d", queue[rear]);
    }
}}
```



The screenshot shows a web-based C compiler interface. The code editor on the left contains the following code:

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = 0, rear = 0;
7
8 void enqueue(int x)
9 {
10     if (rear == MAX)
11     {
12         printf("Queue Overflow\n");
13         return;
14     }
15
16     queue[rear++] = x;
17     printf("Value inserted: %d\n", x);
18 }
19
```

The output window on the right shows the following text:

```
Enter the value: 25
Value inserted: 25

=== Code Execution Successful ===
```

The code has a logical error: in the `enqueue` function, it prints `queue[rear]` after incrementing `rear`. This means it prints the value of the next empty slot in the queue instead of the value that was just inserted. For example, when inserting 25, it prints "Value inserted: 25" because `rear` was 0 and then became 1, so it printed `queue[1]` which was uninitialized (showing 25 in the screenshot).

8. Buggy code in node creation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
int main() {  
    struct Node *newNode;
```

```
    newNode->data = 10;
```

```
    newNode->next = NULL;
```

```
    printf("%d", newNode->data);
```

```
    return 0;
```

```
}
```

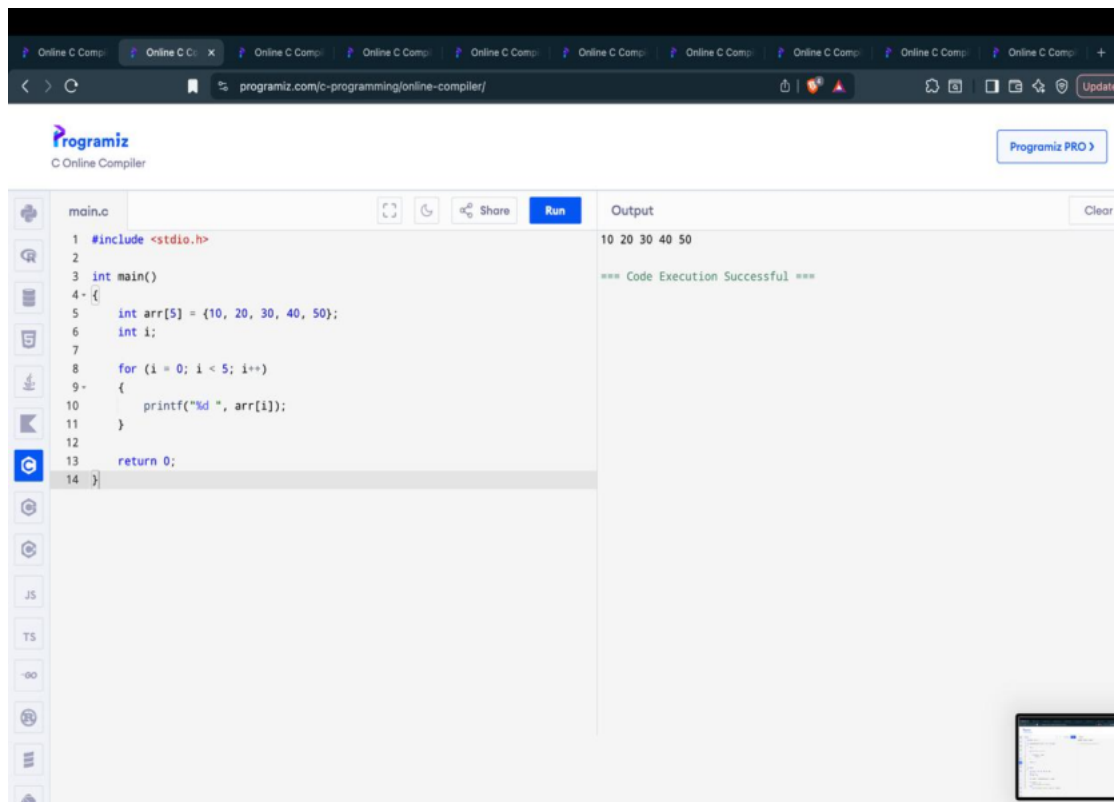
The screenshot displays the Programiz C Online Compiler interface. The code editor on the left contains a C program for binary search. The code defines a `binarySearch` function that takes an array, its size, and a key. It uses a loop to find the index of the key. The `main` function initializes an array `arr` with values {10, 20, 30, 40, 50}, sets `n = 5` and `key = 30`, and calls `binarySearch`. It prints the result or a message if the element is not found. The output window on the right shows the result: "Element found at index 2" and "=== Code Execution Successful ===".

```
main.c  
11  
12     if (arr[mid] == key)  
13         return mid;  
14     else if (arr[mid] < key)  
15         low = mid + 1;  
16     else  
17         high = mid - 1;  
18 }  
19  
20 return -1;  
21 }  
22  
23 int main()  
24 {  
25     int arr[] = {10, 20, 30, 40, 50};  
26     int n = 5;  
27     int key = 30;  
28  
29     int result = binarySearch(arr, n, key);  
30  
31     if (result == -1)  
32         printf("Element not found");  
33     else  
34         printf("Element found at index %d", result);  
35  
36     return 0;  
37 }
```

Output
Element found at index 2
=== Code Execution Successful ===

9. Include missing statements and display the list of elements in linkedlist

```
void display(struct Node *head)
{
while(head != NULL);
{
printf("%d ", head->data);
head = head->next;
}}
```



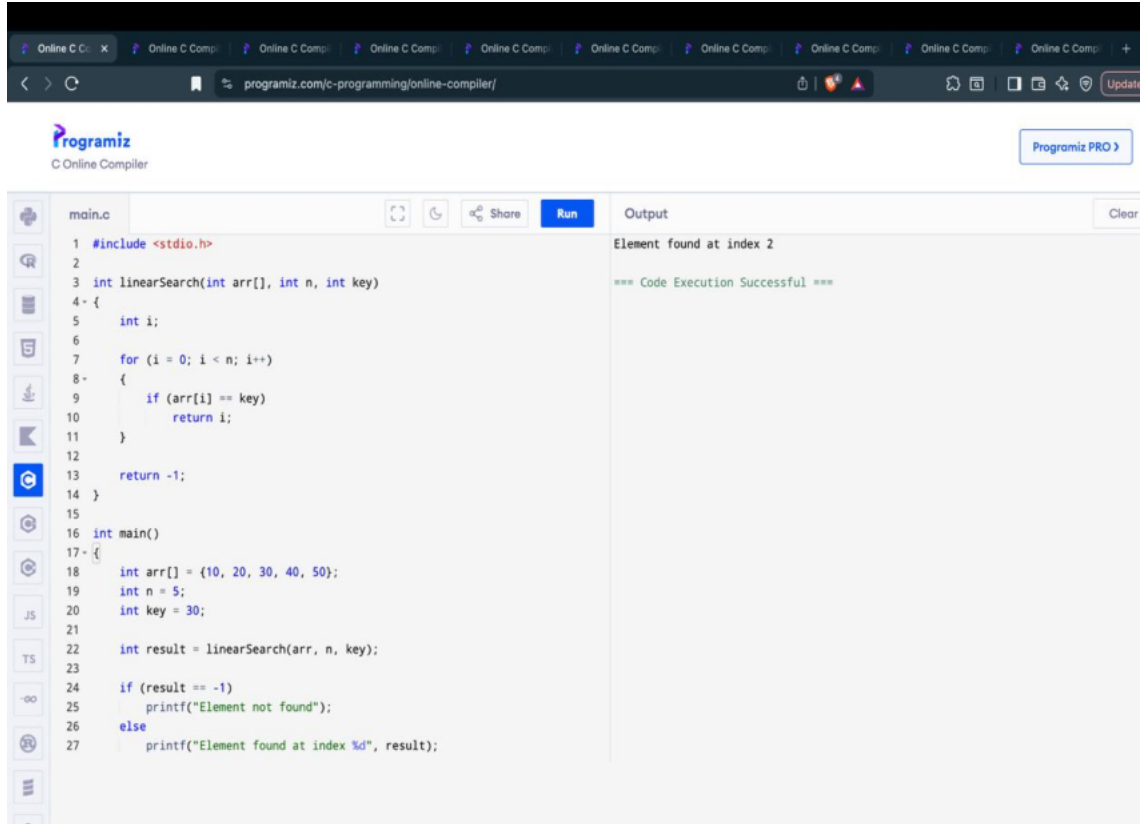
10. Bug in Queue Full Condition-Identify and fix the errors.

```
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item)
{
    if(rear == SIZE-1)
    {
        printf("Queue Full");
        return;
    }

    if(front == -1)
        front = 0;

    rear = (rear + 1) % SIZE;
    queue[rear] = item;
}
```



The screenshot displays the Programiz Online C Compiler interface. The browser address bar shows the URL `programiz.com/c-programming/online-compiler/`. The compiler's header includes the Programiz logo and a "Programiz PRO" button. The main editor area, titled "main.c", contains the following C code:

```
1 #include <stdio.h>
2
3 int linearSearch(int arr[], int n, int key)
4 {
5     int i;
6
7     for (i = 0; i < n; i++)
8     {
9         if (arr[i] == key)
10            return i;
11     }
12
13     return -1;
14 }
15
16 int main()
17 {
18     int arr[] = {10, 20, 30, 40, 50};
19     int n = 5;
20     int key = 30;
21
22     int result = linearSearch(arr, n, key);
23
24     if (result == -1)
25         printf("Element not found");
26     else
27         printf("Element found at index %d", result);
28 }
```

On the right side, the "Output" panel shows the result of the program execution:

```
Element found at index 2
=== Code Execution Successful ===
```

Code of Conduct:

I Shaik Ayesha Tabassum (192525074) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Ayesha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation